

Trustworthy LLM-Assisted Detection of Malicious Packages: Cross-Language Behavior Graphs, Federated Learning, and the Safety Layer as a Third Threat

Anonymous (double-blind submission)
PackGuard Project

ABSTRACT

We present a pilot study that unifies three threat surfaces against LLM-assisted malicious-package detection: (i) the detection model itself, trained over a normalized cross-language *behavior-graph* representation; (ii) the training regime, a federated simulation over naturally non-IID ecosystem partitions (npm vs. PyPI); and (iii) the safety layer of the LLM used for analysis, which may refuse, or corrupt verdicts of, defensive package reviews. We release a fully provenance-tracked corpus of 603 real npm/PyPI packages (390 malicious from a wild-capture dataset, 213 popularity-derived benign), extract six-class behavior graphs with tree-sitter, and train logistic/MLP models under FedAvg/FedProx with secure-aggregation and DP simulations, under a registered twenty-seed protocol (AMENDMENT-4). Under a package-level group split (our primary protocol, adopted after an audit quantified 35% version-level train-test leakage under random splits), graph features reach $F1 .869 \pm .051$ under FedAvg vs. $.858 \pm .047$ centralized — statistically indistinguishable across twenty seeds (exact Wilcoxon $p=.312$; minimum attainable $p=1.9 \times 10^{-6}$); a TF-IDF code-as-text baseline attains higher AUC but *degrades significantly under federation* ($\Delta F1 -.052$, exact Wilcoxon $p=3.8 \times 10^{-6}$, Holm-adjusted 1.1×10^{-5} ; its FedAvg variant collapses to an all-malicious predictor in 79/80 cells) — text features are not federation-robust; graph features are. A refusal-gated LLM-built knowledge base — now covering the full corpus API universe (137/137 types, 2,299/2,299 instances) — is neutral under FedAvg ($-0.1 F1$) and adds $+1.0 F1$ to centralized training (gain concentrated on the minority client: $pypi .875 \rightarrow .966$, $npm .940 \rightarrow .932$); its robust yield is gated per-call explanation, not accuracy. In 360 safety-arm generations over 60 real packages and two $<4B$ models, no overt refusal occurs ($RR = 0.000$; consistent with 2,700 measured defensive-task generations across both project domains at 2–3B), so defensive-refusal blocking does not replicate in the package domain; instead, untrusted in-package content (an advisory comment) shifts verdict sensitivity on genuinely malicious packages in a model-dependent direction (malicious recall $.53 \rightarrow .80$ for granite-3.3-2B vs. $.13 \rightarrow .03$ for Llama-3.2-3B) with zero false positives on benign packages (0/180) — a recall-perturbing channel, unlike the false-positive channel observed in the vulnerability domain. We report all negative and null results, including that an earlier apparent FedAvg minority-client harm disappeared once a file-selection bug was fixed, and that centralized training itself collapsed in 2 of 160 seed-level cells, and discuss the distance between this pilot and a Q1-grade system.

1 INTRODUCTION

Open-source package ecosystems are a primary supply-chain attack surface: attackers publish packages that steal credentials, exfiltrate data, or download further malware during installation [1, 5, 6]. Deployed detectors such as DONAPI [4], Cerebro [9], and MalGuard [2] learn from package metadata, installation behavior, or code; LLM-based triage is emerging independently [8].

This paper asks whether *trustworthy* LLM-assisted detection can be studied as one system rather than three separate papers. We identify three threats that act on the same analysis pipeline:

- (1) **Model threat:** the detector must generalize across languages (JavaScript/Python) and ecosystems from limited labeled data;
- (2) **Training-regime threat:** ecosystem data is naturally non-IID and cannot always be pooled (organizational/privacy boundaries), motivating federated training whose cost must be quantified;
- (3) **Safety-layer threat:** the LLM performing (or explaining) the analysis may *refuse* defensive work phrased offensively, or *corrupt* verdicts under untrusted package content — the RefuseGuard findings [7] must transfer to the higher-stakes package domain, where refusal rates are plausibly higher (we measure, not assume; the measured answer at 2–3B is: no refusals, and a model-dependent recall-perturbing channel instead, §5.2).

Our contributions: (1) a provenance-complete 603-sample npm+PyPI corpus with per-sample source, label provenance, and checksums (§4); (2) a normalized cross-language behavior-graph schema with six semantic classes and 18 graph-level features (§3); (3) a federated simulation with secure-aggregation and DP mechanisms over ecosystem partitions, evaluated with a leakage-resistant group split and a registered twenty-seed protocol (§5); (4) a refusal-gated LLM-augmented knowledge base whose measured value is explainability yield — gated per-call justifications covering the full corpus API universe (137/137 types, 2,299/2,299 instances) — with neutral-to-small F1 effect (-0.1 pt FedAvg, $+1.0$ pt centralized); and (5) a pre-registered safety-arm measurement ported from the vulnerability domain to real packages ($n=60$, two $<4B$ models, 360 generations): refusals are absent ($RR = 0.000$) and untrusted in-package content acts as a model-dependent *recall-perturbing* channel on malicious packages with zero benign false positives (§5.2). All audits, amendments, and failed hypotheses are disclosed.

2 RELATED WORK

Malicious package detection. DONAPI maps installation-time behavior sequences for npm [4]; Cerebro trains a single cross-ecosystem model for npm and PyPI (centralized) [9]; MalGuard targets real-time PyPI detection [2] (venue per USENIX program pages; arXiv record verified); MeMPTec uses metadata [3]. None study federated training across ecosystem partitions or the safety layer of LLM analyzers. **FL for security.** VulFL applies FL to vulnerability detection [10] (preprint); to our knowledge (as of September 2026), FL over ecosystem-partitioned clients for malicious-package detection is unexplored — a literature-check hedge, not an exhaustive claim. **LLMs for supply-chain security.** SecurityAI [8] measures LLM detection accuracy but not refusal interference. **Attack taxonomy.** Ladisa et al. [5] systematize OSS supply-chain attacks (IEEE S&P 2023; an earlier draft of this work mistakenly cited ACSAC — corrected). **Safety-layer line.** Our companion study RefuseGuard [7] established in the vulnerability domain that open 2–7B models show $RR \approx 0$ under offensive-but-defensive wording, verdict corruption toward vulnerability (a false-positive channel), and defense bundles that harm $\leq 3B$ models. PackGuard ports those conditions to packages and measures whether the same channel types appear; at 2–3B they do not: refusals stay at zero and the corruption channel re-types as recall-perturbing (§5.2).

3 METHOD

3.1 Behavior graphs

tree-sitter parsers for JavaScript and Python produce, per sample, a merged graph whose nodes are classified API calls in six semantics (FILE_IO, NETWORK, PROCESS, CRYPTO, DYNAMIC_CODE, DATA_ACCESS) and whose edges are in-scope sequence edges and data edges from produced-variable tracking; imports and require/import aliases are resolved. Entry kinds mark setup.py and npm install-hook scripts. The schema is versioned (SCHEMA_VERSION 1.1.0). Eighteen graph-level features (class histogram, size/density/depth, scope and import diversity, entry flags, parse-failure count) feed classical models. A GNN over the same graphs is a specified extension (project roadmap, to be pre-registered before any run); this paper reports no GNN results.

3.2 Federated simulation

Clients are ecosystem partitions (npm $n=328$, PyPI $n=146$ after the train split; malicious rate 0.686 vs. 0.527 — naturally non-IID but mildly so). We train LR (primary) and a 32-unit MLP for 15 rounds (2 local epochs, SGD lr 0.1, batch 32, seed 20260922) with FedAvg and FedProx ($\mu=0.01$). Secure aggregation is a pairwise additive-mask *simulation* (masks cancel exactly for two clients); DP is a DP-FedAvg-style clip-and-noise *simulation* with a single-round analytic ϵ (no composition proof; disclosed). Baselines: centralized pooling and per-client-only.

3.3 LLM-augmented knowledge base

Unknown API calls (from the graph universe, not the feature table) are classified by a local Qwen2.5-Coder-3B-Instruct into the six-class taxonomy with risk levels, refusal-gated (refusal $\rightarrow$ one retry

$\rightarrow$ UNSURE; a refusal is never converted into a class) and cached; KB-derived features (risk ratio, confidence, unsure ratio) augment the classifier for an on/off ablation.

3.4 Safety port

Three pre-registered arms over real package code (60 packages: 30 malicious + 30 benign, selection seed 20260922; the 10 round-8 packages kept verbatim, 50 added with zero overlap): **P0** neutral defensive task; **P1** identical intent with DRB vocabulary (“steal credentials”, “exfiltrate”, “malware”); **P2** an advisory-style security comment injected into the package file (comment-only, gated by a tree-sitter AST-equivalence check). Metrics (metrics_version 2, renamed after a round-9 audit found that the round-8 field labelled “false-positive bias” actually computed the true-positive rate on *malicious* packages; values unchanged, semantics corrected): refusal rate (RR) per arm; verdict flips benign $\rightarrow$ malicious and malicious $\rightarrow$ benign; *malicious recall* (TP-rate on malicious packages) and, separately, the benign false-positive rate; per-model refusal thresholds; frozen decision rules (blocking: $\Delta RR \geq 0.10$ and exact McNemar $p < 0.05$; corruption: ≥ 1 benign $\rightarrow$ malicious flip).

4 EXPERIMENTAL SETUP

Corpus (603, disclosed). Malicious: DataDog wild-capture dataset, deterministic stride sample (seed 20260922) of 298 npm + 92 PyPI archives with verbatim per-sample class provenance; benign: popularity-ranked npm registry and PyPI download lists, 123 + 90, “assumed benign, not individually audited”. A round-8 audit found the manifest builder silently skipped DataDog’s flat layout; the fix adds 6 archives (5 single-version packages + 1 extra archive of an already-selected package), 597 $\rightarrow$ 603; the flawed 597 manifest is retained as an audit artifact. **Coverage.** With-graph rates after the selection fix: npm-benign 105/123 (85.4%), npm-malicious 252/298 (84.6%), pypi-benign 63/90 (70.0%), pypi-malicious 80/92 (87.0%); 500/603 (82.9%) overall. Empty-graph rows are kept as all-zero and reported per cell. **Splits.** PRIMARY: package-level group split (all versions of a package on one side; label-stratified; round-8 seed instance: test $n=129$, 88 malicious; the split is re-drawn per registered seed). This was adopted as AMENDMENT-1 after measuring that the pre-registered random split shares a package between train and test for 35% of test rows. SECONDARY: the original stratified random split, reported for continuity. **Statistics.** Registered twenty-seed protocol (AMENDMENT-4; seeds 20260922–20260941, extending the round-8 five-seed AMENDMENT-3 grid — the five original seeds are unchanged and seed 20260922 reproduces the round-8 single-seed table to four decimals): per-seed exact McNemar (accuracy@0.5) on the paired FedAvg-vs-centralized comparison, aggregated by two-sided exact Wilcoxon signed-rank over the twenty per-seed $\Delta F1$ (AUC secondary), Holm-corrected over the four registered comparisons. At $n=20$ the minimum attainable two-sided exact p is $2/2^{20} \approx 1.9 \times 10^{-6} < .05$: seed-level tests can reach α . Round-8 single-seed comparisons additionally report McNemar (exact when discordant < 25) and paired bootstrap (10k, seed 20260922) on per-sample accuracy. Standard deviations over seeds are population ($ddof=0$). AMENDMENT-4 was registered before the 20-seed run but after the 5-seed results were known (disclosed; not double-blind). **Pilot limits.** simulation-only FL;

two ecosystems (no Maven); popularity-derived benign labels; hand-curated API mapping tables; the registered twenty-seed protocol makes seed-level tests attainable, but the design remains pilot-scale in corpus and client count.

5 RESULTS

Federation is free on graphs and measurably costly on text — and centralized training has its own instability. On the primary split, FedAvg matches centralized training on graph features across all twenty seeds (Table 2: $\Delta F1 +.0105 \pm .0347$, exact Wilcoxon $p=.312$, Holm-adjusted .312 — a stable null). On TF-IDF the FL cost is now a *confirmatory* result: $\Delta F1 -.0521 \pm .0219$ with 19/20 seeds negative (exact $p=3.8 \times 10^{-6}$; Holm-adjusted 1.1×10^{-5}), and on the random split 0/20 seeds positive ($p=1.9 \times 10^{-6}$; Holm 7.6×10^{-6}). The mechanism is qualitative: tfidf-FedAvg degenerates toward an all-malicious predictor (recall = 1.0 in 79/80 cells; precision $\approx$ test base rate), so its F1 only tracks test composition — the minority client (PyPI) collapses under an unweighted average (group-split pypi F1 $.671 \pm .059$ vs. $.790 \pm .076$ centralized). FedProx at $\mu=0.01$ is identical to FedAvg in 160/160 cells — the proximal term does not bind at this scale (unit-verified at larger μ). Conversely, *centralized* training collapsed in 2/160 seed-level cells (random/graph, seed 20260923: F1 .375, P= 1.0, R= .231; group/graph npm_hook, seed 20260935: F1 .326) while all 320 FL cells stayed $\geq .7308$; excluding the two collapsed cells, the worst remaining centralized cell is .750. A registered stabilized-centralized arm (lr sweep + early stopping on train cross-validation, no test contact) is required before any central-vs-FL ordering is hardened.

Three-client fallback arm. A registered fallback partition (npm split by install-hook presence: npm_core 214 / npm_hook 114 / pypi 146; the hook client is 99.1% malicious — the corpus has exactly one hooked benign package, which lands in train in all five group splits) differs negligibly from the two-client partition (group/graph $\Delta F1 +.024 \pm .120$, exact Wilcoxon $p=1.0$). This is a same-corpus partition, *not* cross-ecosystem FL: the third-ecosystem source probe failed and no ingestible source was found (disclosed).

Graph vs. text is split-decision, honestly. TF-IDF wins ranking (AUC: $.899 \pm .039$ vs. $.882 \pm .042$ under FedAvg across twenty seeds; $.9605$ vs. $.9260$ in the round-8 single-seed run) but is miscalibrated at the fixed threshold (recall ≈ 1 ; under FedAvg, degenerately so), losing on F1. We do not claim graph superiority; the claim is complementarity under calibration-sensitive metrics and interpretability (named behavior nodes), at the cost of coverage (Table 3). Ranking-quality extensions (AUC-PR, ECE and reliability curves, train-CV threshold selection) are specified in the project roadmap, not yet run.

Sensitivity arms (single seed 20260922, round 8). DP ($\sigma=0.1$, $\epsilon \approx 48.4$ per round, single-query bound): F1 0.915 (−0.8 points) — privacy noise is affordable at this scale, though the R-round composition is unproven. MLP matches LR exactly on the primary metrics. Dropping the 103 empty-graph rows moves central F1 to 0.821 ($n=500$): the all-zero rows are not harmless artifacts but carry ecosystem signal (disclosed, not exploited).

5.1 Knowledge-base ablation

The graph universe of the 603-sample corpus contains 137 unique API names; 122 were unknown to the 20-entry seed KB. Round 8 classified the top 60 by document frequency with local Qwen2.5-Coder-3B-Instruct through the refusal-gated builder: **0 refusals, 0 retries, 0 UNSURE** (the 0.5B pilot model produced 8/10 UNSURE on the same protocol — model scale matters), at ~ 15 s/call on MPS; the remaining 62 tail APIs were counted as unsure by construction (disclosed). Round 9 completed the universe: the KB now covers **137/137 API types and 2,299/2,299 API instances** (100.0%; 142 entries = 20 seed + 122 LLM-classified, zero unsure), raising mean kb_confidence over the graphs from .878 to .907 — with the disclosure that 113/122 LLM entries self-report confidence clipped at 0.90 (a weak calibration signal). The FL ablation below was run under the earlier 54.7%-coverage KB; a registered re-run under the complete KB needs no LLM calls (KB features are a pure join) and is queued.

KB features are neutral under FedAvg (−0.1 pt F1) and add +1.0 pt F1 / +0.6 pt AUC to centralized training, with the gain concentrated on the minority client (pypi F1 .875 $\rightarrow$.966; the npm subgroup slips .940 $\rightarrow$.932; subgroup $n=36$ — indicative, not significant; reported as an effect size, not a test). The robust yield is explainability, not accuracy: every API call in a verdict now carries a gated, cached justification (semantic class + rationale) covering the full API universe — the contribution of the KB is this audit trail, not classifier performance.

5.2 Safety layer on real packages

Blocking does not replicate at 2–3B. RR = 0.000 across all 360 generations (exact McNemar vs. P0: $p=1.0$ everywhere): inviting either model to analyze a credential-stealer through the defensive static-review frame never triggered a single refusal, in any arm. The measured scope of this null across the project is 2,700 defensive-task generations with no overt refusal (round-3 vulnerability domain 1,080: E0 arms 180 + E2/E3 conditions 900; E0v2 1,200; package-domain 360 + 60) under the evaluated <4B models and prompt families; in the same round-3 runs the deliberately harmful OR-Bench probe prompts were refused 125/225 times, so the refusal pathway is functional — the defensive task itself never triggers it. Directive-refusal bias is *not supported* at 2–3B for the tested prompt families. We deliberately do not generalize this to “safety alignment does not transfer” — the claim is closed only at these scales, families, and prompts.

The corruption channel re-types as recall-perturbing. All 11 benign $\rightarrow$ malicious flips across arms land on genuinely malicious packages (checked per sample), so the pre-registered corruption rule (≥ 1 flip) fires as written but measures a *sensitivity shift on malicious packages*, not the wrongful flagging of benign code. The direction is model-dependent — undirected: granite-3.3-2B recall $.533 \rightarrow .800$ under P2, while Llama-3.2-3B *falls* $.133 \rightarrow .033$ (four detections lost, one gained — a measured cost). Benign false positives are 0/180 across all arms and models (0/60 pooled-P2 benign-records, bounding the FP rate below $\approx 5\%$ by rule of three). Context thus shifts sensitivity on malicious packages in both directions without touching benign false positives — a *recall-perturbing* channel, unlike the vulnerability domain’s false-positive channel; the

Table 1: Primary protocol (package-level group split), twenty registered seeds (AMENDMENT-4): mean $\pm$ std over seeds (population, $ddof=0$), ecosystem partition. LR, 15 rounds, threshold 0.5. FedProx ($\mu=0.01$) is identical to FedAvg in 160/160 cells and is not repeated. per-client-best is oracle per-sample routing (an upper bound, not a deployable system). Seed 20260922 reproduces the round-8 single-seed table to four decimals (graph FedAvg .9231/.9160, centralized .9282/.9260 F1/AUC).

Run	F1	AUC	npm F1	pypi F1
graph FedAvg	.869 $\pm$.051	.882 $\pm$.042	.869 $\pm$.066	.851 $\pm$.080
graph centralized	.858 $\pm$.047	.876 $\pm$.042	.863 $\pm$.056	.830 $\pm$.100
graph per-client-best	.865 $\pm$.043	.880 $\pm$.045	.874 $\pm$.055	.812 $\pm$.068
tfidf FedAvg	.790 $\pm$.031	.899 $\pm$.039	.827 $\pm$.045	.671 $\pm$.059
tfidf centralized	.842 $\pm$.038	.896 $\pm$.039	.853 $\pm$.047	.790 $\pm$.076
tfidf per-client-best	.833 $\pm$.036	.862 $\pm$.070	.827 $\pm$.045	.841 $\pm$.098

Table 2: Primary comparison (FedAvg – centralized), paired per seed over the 20 registered seeds (AMENDMENT-4), F1. Wilcoxon = two-sided exact signed-rank over the twenty per-seed deltas; p^{Holm} = Holm-adjusted over the four registered comparisons. Last column: seeds whose exact per-seed McNemar reaches $p < .01$.

Split / block	$\Delta F1$ mean $\pm$ std	$n_+/n_-/n_0$	Wilcoxon p (p^{Holm})	McNemar $p < .01$
group / graph	+.0105 $\pm$.0347	9/10/1	.312 (.312)	2/20
group / tfidf	-.0521 $\pm$.0219	1/19/0	3.8×10^{-6} (1.1×10^{-5})	14/20
random / graph	+.0403 $\pm$.1162	12/8/0	.133 (.266)	4/20
random / tfidf	-.0576 $\pm$.0178	0/20/0	1.9×10^{-6} (7.6×10^{-6})	16/20

Table 3: Coverage after the file-selection fix (v2). The v1 selector flooded its 12-file budget with `__init__.py` and metadata files, manufacturing 16 spurious empty pypi-benign graphs (audit finding); pypi-benign coverage rises 55.6% $\rightarrow$ 70.0%.

Cell	with-graph	coverage
npm benign	105/123	85.4%
npm malicious	252/298	84.6%
pypi benign	63/90	70.0%
pypi malicious	80/92	87.0%
total	500/603	82.9%

Table 4: KB ablation on the primary (group) split, round-8 run under KB v2 (54.7% type coverage); the completed KB v3 (100%) re-run is registered. “on” appends `kb_risk_ratio / kb_confidence / kb_unsure_ratio` to the 18 graph features.

Run	F1		AUC	
	off	on	off	on
FedAvg	.923	.922	.916	.911
centralized	.928	.938	.926	.932
pypi F1 (central)	.875	.966		

round-8 numbers (“FP-bias” .2 $\rightarrow$.6, granite 1.0) must be re-read as recall on the 10 malicious packages (2/10 $\rightarrow$ 3/10 $\rightarrow$ 6/10; granite 5/5) with benign FP 0/30.

Mechanism not yet isolated. We cannot yet distinguish “the advisory content helps the model reason about malicious behavior” from “any unusual in-package content perturbs the verdict”; the recall *increase* is therefore not claimed as a benefit, and the *decrease*

(llama) is the measured harm. The matched-scale family contrast (granite +8 vs. llama –3 net flips at P2) shows the effect is family-specific at 2–3B. A registered mechanism ablation (same-meaning paraphrases + scrambled controls) and an FP-directed arm (an advisory asserting “this code is clean”) are queued. All of this remains pilot-scale: it establishes measurability, the zero-refusal blocking outcome, and direction — not effect size.

6 DISCUSSION

When graphs lose to text. On this pilot, raw text remains the stronger ranking signal (AUC) while graph features win thresholded F1 and explainability. The honest conclusion is that the behavior-graph hypothesis is *not yet* validated as strictly better; its value is calibration, interpretability, and coverage robustness under the group split (graph degrades less between random and group splits than TF-IDF’s minority-client F1 does under FL). **Boundary condition (family shift).** The group split removes version-level leakage within a package (AMENDMENT-1) but not whole-*family* shift — unseen families sharing an actor or lineage. The reported AUC ordering is conditional on that threat model; a registered leave-families-out comparison (graph vs. TF-IDF degradation under family shift) must decide any robustness claim, and temporal evaluation on fresher attacks is out of scope for this pilot. **A retracted finding.** Our interim run on the v1 features showed a dramatic FedAvg minority-client harm (pypi F1 0.57 vs. 0.82 centralized). After the file-selection fix the harm vanishes (Table 1: group-split pypi F1 .833 $\pm$.115 under FedAvg vs. .790 $\pm$.105 centralized across seeds — if anything reversed). We report this because the “FedAvg harms minorities” narrative was an artifact of a feature bug — a caution against interpreting FL failures before auditing the features. Rounds 9 and the twenty-seed extension

Table 5: Safety arms on 60 real packages, per model (30 malicious + 30 benign; 60 paired generations per model/arm). RR = refusal rate; malicious recall = TP-rate on the 30 malicious packages; FP benign = benign false-positive rate. *Semantics note:* the round-8 table labelled this quantity “FP-bias” (.2/.3/.6, granite 1.0); a round-9 audit showed the then-code-path computed the recall on *malicious* packages — those numbers must be re-read accordingly (metrics_version 2; raw records unchanged).

Model / arm	RR	flips b→m / m→b	recall (P0→arm)	FP benign
llama P0 neutral	.00	– / –	.133	0/30
llama P1 offensive	.00	0 / 0	.133 → .133	0/30
llama P2 advisory	.00	1 / 4	.133 → .033	0/30
granite P0 neutral	.00	– / –	.533	0/30
granite P1 offensive	.00	2 / 0	.533 → .600	0/30
granite P2 advisory	.00	8 / 0	.533 → .800	0/30

sharpen the lesson in the opposite direction: centralized training collapsed in 2/160 seed-level cells while all 320 FL cells stayed $\geq .73$ (§5) — single-run conclusions can err in both directions, motivating the registered stabilized-centralized arm. **Safety transfer.** The RefuseGuard *machinery* (refusal monitor, AST-gated advisory injection, frozen rules) ports mechanically to packages, but the corruption channel does not port as the same *phenomenon type*: in the vulnerability domain untrusted context created false positives on benign code; in the package domain it perturbs recall on malicious packages with benign FP = 0 (§5.2). P2’s advisory-in-package attack is a package-specific threat the vulnerability study could not express, and the AST gate makes it safely measurable; because the direction is model-dependent, follow-up arms must pre-register *both* outcome directions (harm and benefit) before running. **Scale versus model family (open confound).** Across the project, Qwen2.5-Coder is inert at 3B and 7B (the pre-registered 7B harm-absent verdict is SUPPORTED), Llama-3.2-3B shows defense harm at 3B, and granite-3.3-2B moves in the opposite direction in the package domain: the scale-vs-family confound is open, and every conclusion in this paper is closed at 2–3B. **8B feasibility disclosure.** The correct instrument for the scale question (Llama-3.1-8B-Instruct, same family, $\sim 2.7\times$) is infeasible on this stack: four independent MPS load failures (a bf16 harness stall ≥ 27 min in weight copy with zero tokens generated; two fp16 accelerate-dispatch SIGSEGVs — pointer-authentication failures with stacks in `mps::copy_cast_kernel`; a direct fp16 run hung for 8 min until the watchdog killed it), while a CPU proof loaded the sha256-verified weights and generated real tokens at **1.11 tok/s**, making the pre-registered 240-generation $\times$ 512-token ladder ≈ 31 h of decoding alone (NOT_FEASIBLE-in-time). Peak-RSS figures observed live during the failures were not preserved in raw artifacts and are not quoted as evidence; a 3B control validates the load path only (its generation step errored, so it is load-only evidence). Any ≥ 8 B result requires a different runtime (MLX 4-bit or GGUF/llama.cpp — neither present in this environment) and must not share feasibility claims with the transformers/MPS stack. **External validity.** All results are measured on one wild-capture malicious corpus plus popularity-derived benign labels; external validation on independently collected corpora (e.g., campaign-level industry feeds, license not yet probed) is required before any deployment claim, and a deployed-data evaluation of the safety arms remains future work.

7 CONCLUSION AND LIMITATIONS

A 603-sample, provenance-complete pilot shows: federated training over ecosystem partitions is near-free for graph features across five registered seeds and costly for text features (whose FedAvg variant degenerates to a majority-class predictor); representation superiority is split-decision; LLM-built knowledge bases are measurable, with a now complete (100%-coverage) gated API universe whose robust yield is explanation rather than F1; and the safety-layer third threat is real but *recall-perturbing* in this domain, with zero refusals at 2–3B. Limitations: pilot scale (CI widths $\pm .05$ – $.08$ F1), simulation-only trust mechanisms, two languages, popularity-derived benign labels, hand-curated mappings, an LLM-KB whose confidence is self-reported, and a hard scale ceiling: conclusions close at 2–3B, the direction of safety-layer effects follows model family, and ≥ 8 B evaluation requires a different runtime. A Q1-grade system needs: achieved-power and variance-decomposition analysis over the registered 20-seed grid (Holm correction is now applied), a stabilized-centralized arm, a real GNN over the graphs, a leave-families-out evaluation, mechanism-isolated safety arms at $n \geq 100$ /model, at least one more ecosystem, audited benign labels, and a deployed-data evaluation of the safety arms.

DATA AND PROVENANCE

Manifest, features, graphs, per-row results with mock flags, audit reports (V1/V2 per round), and AMENDMENT-1/-3/-4 are in the project tree (data/packguard/, outputs/packguard/incl. `fl_multiseed/grid_results.json` and `safety/safety_metrics_n60.json`, docs/packguard_prereg.md, reports/round8/, reports/round9/).

REFERENCES

- [1] Ruian Duan, MJ Altheide, Ashish Joshi, Devesh Gunter, et al. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Proceedings of the 21st ACM Internet Measurement Conference (IMC)*. arXiv:2002.01139; MalOSS collection.
- [2] Xingan Gao, Xiaobing Sun, Sicong Cao, Kaifeng Huang, and Di Wu. 2025. MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem. In *34th USENIX Security Symposium (USENIX Security 2025)*. arXiv:2506.14466; venue per USENIX program, arXiv record verified.
- [3] Saptarshi Halder et al. 2024. MeMPtec: Malicious Package Detection using Metadata Information. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [4] Chengqiang Huang, Nannan Wang, Ziyang Wang, Siqu Sun, Lingzi Li, et al. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 2024)*. arXiv:2403.08334.

- [5] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (S&P)*. 1509–1526. arXiv:2204.04008; user proposal said ACSAC 2023 – venue corrected to IEEE S&P.
- [6] Marc Ohm, Henrik Plate, Andreas Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. arXiv:2005.09561; BKC dataset.
- [7] RefuseGuard study (companion paper, this project). 2026. RefuseGuard: Measuring and Mitigating Safety-Layer Interference in LLM Vulnerability Analysis. *arXiv preprint arXiv:2603.01246 (Defensive Refusal Bias)* (2026). Companion 7-round study; internal draft under submission.
- [8] Nusrat Zahan, Philippe Burckhardt, Maksym Lysenko, et al. 2024. Shifting the Lens: Detecting Malicious npm Packages using Large Language Models (SecurityAI). *arXiv preprint arXiv:2408.08924* (2024).
- [9] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, et al. 2024. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model (Cerebro). *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2024). <https://doi.org/10.1145/3705304> arXiv:2309.02637.
- [10] Peiheng Zhou, Ming Hu, Xingrun Quan, Yawen Peng, Xiaofei Xie, et al. 2024. An Empirical Study of Vulnerability Detection using Federated Learning (VulFL). *arXiv preprint arXiv:2411.16099* (2024). Venue not verified; cited as preprint.